ЛЕКЦІЯ 3
ОСНОВИ МАШИННОГО НАВЧАННЯ ТА НЕЙРОННИХ МЕРЕЖ ДЛЯ ВИЯВЛЕННЯ КІБЕРАТАК

ЗМІСТ

1. Вступ
2. Основи машинного навчання та нейронних мереж
3. Структура нейронних мереж
4. Процес навчання нейронних мереж
5. Основні параметри навчання нейронних мереж
6. Висновки
7. Питання для самоконтролю

1. ВСТУП

Сучасні комп'ютерні мережі генерують величезні обсяги даних. Кожна мережева взаємодія (network connection) створює численні записи: IP-адреси, порти, протоколи, обсяг переданих даних, час з'єднання, стан TCP-з'єднання тощо. У великих мережах кількість таких записів може становити мільйони або навіть мільярди подій на день.

Традиційний підхід до виявлення атак, що передбачає ручний аналіз або використання сигнатур, має обмеження. До проблем класичних систем виявлення атак належать необхідність обробки величезного обсягу даних, постійна поява нових атак, складність сучасних атак та їхня швидкість.

Для подолання цих викликів у кібербезпеці активно застосовується машинне навчання (Machine Learning). Машинне навчання — галузь штучного інтелекту, що дозволяє комп'ютерним системам автоматично виявляти закономірності у даних та робити прогнозування без явного програмування правил. До основних типів машинного навчання належать:

1. **Кероване навчання (Supervised Learning)** — метод машинного навчання, при якому модель навчається на даних, що мають вхідні параметри (features) та правильні відповіді (labels). Приклад даних мережевих подій, які можуть застосовуватися для керованого навчання подано у таблиці 1.

duration	protocol	src_bytes	dst_bytes	label
10	tcp	1200	800	normal
0	tcp	5000	0	neptune

Табл. 1. Приклад даних, що використовується для керованого машинного навчання

У керованому типі машинного навчання модель навчається знаходити залежність «ознака → відповідь» (features → label). У нашому курсі використовується саме кероване машинне навчання.

2. **Некероване навчання (Unsupervised Learning)** — метод машинного навчання, при якому правильних відповідей немає. Алгоритм сам намагається знайти аномалії, кластери та структуру даних. Такий підхід використовується для виявлення нових атак та кластеризації мережевого трафіку.

3. **Навчання із підкріпленням (Reinforcement Learning)** — метод машинного навчання, що використовує так звану «систему нагород і штрафів». Використовується у автономних системах, робототехніці та ігровому штучному інтелекті. У кібербезпеці застосовується значно рідше.

Однією з основних сфер застосування машинного навчання у кібербезпеці є система виявлення вторгнень (**Intrusion Detection Systems, IDS)** — система, що аналізує мережеву або системну активність з метою виявлення спроб несанкціонованого доступу або атак. Основні типи таких систем наступні:

1. Системи на основі сигнатур, що працюють за принципом порівняння із відомими шаблонами атак. Наприклад, якщо мережевий пакет містить характерний шаблон певної атаки, скажімо, SQL Injection, то система сигналізує про спробу зловмисної дії. Серед недоліків таких систем варто виділити залежність від бази сигнатур та невміння виявляти нові типи атак.

2. Системи на основі аномалій, при якій модель вивчає нормальну поведінку мережі та виявляє відхилення від норми. Такі системи можуть виявляти нові типи атак та адаптуватися до змін мережі**.** Саме цей підхід реалізується у цьому курсі.

2. ОСНОВИ МАШИННОГО НАВЧАННЯ ТА НЕЙРОННИХ МЕРЕЖ

Ідея машинного навчання відносно проста і полягає у навчанні моделі на прикладах замість написання правил. В якості таких правил використовується набір даних, що прийнято називати датасетом (dataset).

Датасет — це структурована таблиця даних, що використовується для навчання моделей. В якості рядків такої таблиці використовується запис про те чи інше мережеве з'єднання (подію), а в якості колонок — її ознаки, тобто числові або категоріальні параметри.

Наприклад, у популярному датасеті NLS-KDD використана 41 ознака, серед яких: тривалість з'єднання (*duration*), тип протоколу *(protocol_type*), сервіс (*service*), прапорець (*flag*), кількість байтів у запиті клієнта (*src_bytes*), кількість байтів у відповіді сервера (*dst_bytes*) та інші.

Окрім вищезгаданих у цьому датасеті присутня колонка, що містить мітки класу (label) для кожної події. Класом мережевого з'єднання є висновок системи, чи була подія безпечною (normal) або ж атакою (класи атак що належать до категорій DoS, Probe, R2L або U2R).

Якщо модель тестується на тих самих даних, на яких навчалася, вона може просто запам'ятати їх. Таке явище називається перенавчанням (overfitting). Тому перед безпосереднім навчанням моделі датасет обов'язково ділиться на дві частини: тренувальну та тестову (водночас, датасет NSL-KDD вже поширюється у двох варіантах, тому його розділення не потрібне).

Тренувальна частина датасету використовується для навчання моделі, яка на вході отримує ознаки та мітку класу мережевої події.

Тестова ж частина використовується для перевірки якості такої моделі. На її вхід подаються лише ознаки мережевої події, а очікуваним результатом її роботи є надання відповіді щодо мітки її класу.

Одним із методів машинного навчання є **нейронні мережі.**

Нейронна мережа — математична модель, натхненна структурою біологічної нервової системи. Від інших алгоритмів машинного навчання вона відрізняється тим, що складається з: а) великої кількості штучних нейронів; б) зв'язків між ними; в) ваги кожного зв'язку**.** Вагою зв'язку вважається математична характеристика його важливості для результату функціонування нейронної мережі.

Нейрон — це обчислювальний елемент нейронної мережі, що полягає у математичній функції, яка отримує кілька числових значень на вході, комбінує їх та передає результат далі. Наприклад, три ознаки мережевого з'єднання *src_bytes*, *dst_bytes* та *duration* нейрон отримає як x_1, x_2 та x_3 відповідно.

Зв'язок — це канал передачі інформації, що поєднує кожен вхідний параметр із нейроном. Найважливішою характеристикою цього зв'язку є його вага.

Вага — це коефіцієнт, який визначає, наскільки важливим є певний зв'язок для нейрона. Наприклад, ваги трьох вищезгаданих параметрів отримають значення w_1, w_2 та w_3, а важливість кожного параметра буде знаходитись у прямій пропорційній залежності від значень цих вагів. Саме у процесі налаштування ваг нейронних зв'язків полягає процес навчання моделі.

В контексті кібербезпеки метою нейронної мережі є **знаходження закономірності між ознаками і класами атак.** Наприклад, мережа може визначити, що поєднання дуже великого значення *src_bytes*, нульового *dst_bytes* та маленького *duration* часто означає **DoS атаку**.

Бажаним кінцевим результатом навчання моделі є її можливість працювати із іншим набором даних, на яких вона не тренувалася. Таким чином, після навчання на тисячах прикладів атак модель зможе визначати нові атаки у реальному мережевому трафіку.

3. СТРУКТУРА НЕЙРОННИХ МЕРЕЖ

Взаємопов'язані нейрони організовуються у **шари (layers)**. Класична архітектура нейронної мережі включає три основні типи шарів, операції на яких відбуваються послідовно:

1. Вхідний (input layer).
2. Приховані (hidden layers).
3. Вихідний (output layer).

Важливо розуміти, що кількість нейронів N на кожному шарі визначається різними критеріями. Для вхідного шару кількість нейронів відповідає кількості n досліджуваних ознак $x_1 \dots x_n$. На прихованому шарі кількість нейронів визначається експериментально, з тією логікою, що більша кількість нейронів дозволить дослідити складніші залежності, але сповільнить процес навчання. На вихідному шарі кількість нейронів відповідає кількості можливих результатів роботи нейронної мережі (у випадку датасету NLS-KDD така кількість дорівнює 5: *Normal, DoS, Probe, R2L, U2R*).

Окрім того, кількість зв'язків (входів) кожного нейрона на наступному шарі дорівнює кількості нейронів на попередньому шарі.

3.1. Вхідний шар (Input Layer)

Вхідний шар приймає дані ззовні та передає їх далі в мережу. На цьому шарі кожен нейрон відповідає **одній ознаці (feature)** вхідних даних. При цьому безпосередньої обробки даних не відбувається, а натомість має місце передача даних далі на прихований шар. Таким чином, на вхідному шарі нейрони є просто контейнерами для даних, а математичні операції не проводяться.

В загальному прикладі нейронних мереж, при обробці зображення розміром 28×28 пікселі було б створено $N_1 \dots N_{784}$ нейрони. В задачах кібербезпеки при аналізі датасету із 10 ознаками створюється $N_1 \dots N_{10}$ нейронів.

Кожен із нейронів передає значення своєї ознаки далі на прихований шар.

3.2. Приховані шари (Hidden Layers)

Ідеєю обробки значень на приховану шарі є дослідження складних залежностей між отриманими від вхідного шару параметрами.

На прихованому шарі створюються нейрони у кількості, що визначається експериментально людиною, що проводить навчання нейронної мережі. Кожен створений нейрон створює зв'язки із **усіма** отриманими від вхідного шару параметрами.

Після цього для кожного нейрону виконуються дві послідовні ключові операції: лінійна комбінація та активаційна функція.

3.2.1. Лінійна комбінація (зважена сума)

Для кожного нейрону обчислюється зважена сума добутків вхідних параметрів і їхніх ваг. Математично ця залежність може бути виражена у наступній формулі:

$$Y = x_1 w_1 + x_2 w_2 + x_3 w_3 + \dots + x_n w_n + b,$$

де Y — зважена сума нейрона прихованого шару, x_1, x_2, x_3 — перші три параметри, w_1, w_2, w_3 — ваги відповідних зв'язків, n — кількість досліджуваних параметрів, b — зміщення.

Значення зміщення b при цьому використовується для того, щоб нейрон мав ненульовий результат навіть при нульових значеннях параметрів.

Результатом цього кроку є отримання значення Y для кожного нейрона N, яким були властиві різні значення ваг $x_1 \dots x_n$.

3.2.2. Активаційна функція

Важливо розуміти, що отримані внаслідок лінійної комбінації значення є числами, що мають лише абстрактний математичний сенс. Для виправлення цієї ситуації застосовується одна **із активаційних функцій**.

Активаційна функція (activation function) — це математична функція, яка застосовується до лінійної комбінації входів нейрона для формування **вихідного сигналу нейрона**:

$$a = f(Y),$$

де a — активаційна функція нейрона, а Y — його зважена сума.

Необхідність застосування активаційної функції аргументується наступними факторами:

1. Забезпечення нелінійності — без активаційної функції нейронна мережа була б просто лінійним перетворенням входів. Нелінійна активація дозволяє мережі аналізувати складні шаблони і залежності між ознаками.
2. Обмеження або нормалізація значень — деякі активаційні функції (*sigmoid, tanh, softmax*) обмежують вихід у певному діапазоні, що важливо для стабільності навчання і інтерпретації результатів.
3. Прискорення процесу навчання — вибір правильної активаційної функції впливає на швидкість функціонування мережі та дозволяє уникнути деяких проблем при визначенні помилки роботи мережі на наступних етапах навчання.

Приклади найбільш популярних активаційних функцій подані у таблиці 2.

Функція	**Формула**	**Діапазон**	**Основне призначення**
ReLU	$f(x) = \max(0, x)$	[0, ∞)	Створення нелінійності, швидке навчання
Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$	(0, 1)	Перетворення на ймовірність, нормалізація значень
Tanh	$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$	(-1, 1)	Центрування виходу навколо 0, створення нелінійності
Softmax	$f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$	[0,1], $\sum_i f(z_i) = 1$	Перетворення вектору виходів у ймовірності класів
Leaky ReLU	$f(x) = \max(0{,}01x, x)$	(-∞, ∞)	Аналогічно ReLU, але виходи нейрони не обнуляються

Табл. 2. Активаційні функції нейронних мереж

Результати активаційної функції нейронів передаються далі на вихідний шар. Водночас, важливо розуміти, що з метою дослідження ще глибших залежностей і шаблонів послідовних прихованих шарів може бути декілька, в залежності від задач нейронної мережі.

Для прихованих шарів часто використовують активаційну функцію ReLU, бо вона швидка і дозволяє мережі формувати нелінійні послідовності. Sigmoid або tanh застосовують рідше, зазвичай у невеликих або старих мережах.

3.3. Вихідний шар (Output Layer)

Призначенням вихідного шару є формування кінцевого результату роботи нейронної мережі за рахунок створення вектору ймовірностей.

Кількість нейронів у вихідному шарі має відповідати кількості можливих результатів мережі (як вже згадувалося, у випадку датасету NLS-KDD їх 5). Відповідно, кількість входів (зв'язків) кожного нейрона на вихідному шарі дорівнює кількості нейронів на прихованому шарі.

На вихідному шарі застосовуються **ті самі математичні перетворення, що і на прихованому шарі: лінійна комбінація та активаційна функція**. Водночас, варто розглянути деякі особливості.

На відміну від прихованого шару при підрахунку лінійної комбінації використовуються не значення самих ознак, а значення активаційних функцій нейронів, отриманих від попереднього шару. Окрім того, очевидно, що значення ваг нейронів вихідного шару незалежні від ваг нейронів прихованого шару.

$$Z = \sum_{j=1}^{n} f_j w_j + b,$$

де Z — зважена сума нейрона вихідного шару, f_j — значення активаційної функції нейрона прихованого шару, w_j — значення відповідної ваги, b — зміщення

Щодо активаційної функції на вихідному шарі важливо розуміти, що хоч теоретично можна застосувати таку саму функцію, як і на прихованому шарі, але на практиці цей вибір для більшості задач не є оптимальним. Особливо це характерно для багатокласової класифікації, як у випадку датасету NLS-KDD, бо для неї найчастіше застосовується активаційна функція Softmax.

Визначальною характеристикою активаційною функції Softmax є її застосування для всього шару, а не для кожного нейрону окремо.

Softmax дозволяє перетворити значення зважених сум кожного нейрона у вектор ймовірностей:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}},$$

де $\sigma(z)_i$— вихідне значення Softmax для нейрона i, z_i — зважена сума нейрона i.

Наприклад, у випадку NSL-KDD вектор матиме вигляд [P_{Normal}, P_{DoS}, P_{Probe}, P_{R2L}, P_{U2R}], де P — ймовірності кожного класу, обчислені як результат активаційної функції Softmax для відповідного нейрона. Оскільки на вихідному шарі кожен нейрон відповідає можливому класу мережевої події, то передбаченням роботи моделі стає клас із найбільшою ймовірністю.

4. ПРОЦЕС НАВЧАННЯ НЕЙРОННИХ МЕРЕЖ

Алгоритм навчання нейронної мережі передбачає наступні послідовні етапи: ініціалізація значень ваг, пряме поширення, обчислення помилки та зворотнє поширення.

4.1. Ініціалізація значень ваг

На початку процесу нейронної мережі значення ваг $w_1 \ldots w_n$ на прихованому і вихідному шарах приймаються випадковими. На подальших етапах це створить випадкові значення лінійних комбінацій, активаційних функцій і, відповідно, передбачень мережі, але це необхідний крок у процесі навчання.

4.2. Пряме поширення сигналу (forward propagation)

Дані проходять через нейронну мережу у описаній вище послідовності: вхідний шар → прихований шар → вихідний шар. Результатом прямого поширення сигналу є передбачення моделі.

4.3. Обчислення помилки

Зроблене в результаті прямого поширення сигналу передбачення порівнюється із правильною відповіддю. Уявімо, що у вектор ймовірностей [P_{Normal}, P_{DoS}, P_{Probe}, P_{R2L}, P_{U2R}] найбільше значення мала ймовірність P_{DoS}.

В такому випадку обчислення помилки передбачення відбувається наступним чином. Отриманий вектор ймовірностей порівнюється із іншим вектором [Y_{Normal}, Y_{DoS}, Y_{Probe}, Y_{R2L}, Y_{U2R}], в якому всі елементи набувають значення 0, окрім елемента із порядковим номером передбачення мережі, який набуває значення 1.

Помилка мережі обчислюється за наступною формулою, що у сфері нейронних мереж називається крос-ентропійною (cross-entropy):

$$L = -\sum_{i=1}^{K} Y_i \cdot \ln(P_i),$$

де K — кількість класів, Y_i — значення елементу i вектора порівняння (1 відповідає правильному класу, 0 іншим), P_i — передбачена ймовірність для класу i.

Розглянемо конкретний приклад. Уявімо, що вектор ймовірностей після активаційної функції Softmax набув вигляду [0.01, 0.7, 0.2, 0.05, 0.04], відповідно, вектор порівняння має вигляд [0, 1, 0, 0, 0].

Тоді помилка L так:

$$L = -(0 \cdot \ln 0{,}01 + 1 \cdot \ln 0{,}7 + 0 \cdot \ln 0{,}2 + 0 \cdot \ln 0{,}05 + 0 \cdot \ln 0{,}04) = -\ln(0.7) \approx 0{,}357$$

Отримана помилка є характеристикою роботи нейронної мережі, а процес налаштування мережі зводиться до мінімізації значення цієї помилки.

4.4. Зворотне поширення помилки (Backpropagation)

На цьому етапі мережа повинна визначити, які саме ваги призвели до отримання помилки. Алгоритм поширює помилку через мережу у зворотному напрямку: вихідний шар → прихований шар → вхідний шар. Це дає змогу визначити, які нейрони дали найбільший внесок у помилку і значення яких ваг потрібно змінити. Таким чином для кожної ваги обчислюється так званий **градієнт**, чия математична сутність зводиться до похідної $\frac{\partial L}{\partial w}$, де L — обчислена похибка, а w — значення ваги.

4.5. Оновлення значень ваг

Процес оновлення значень вагу машинному навчанні називається градієнтним спуском (gradient descent). Формула градієнтного спуску наступна:

$$w_{\text{нове}} = w_{\text{попереднє}} - \eta \cdot \frac{\partial L}{\partial w},$$

де $w_{\text{попереднє}}$ — попереднє значення ваги, η — швидкість навчання, L — обчислена похибка, а w — значення ваги.

Варто зауважити, що швидкість навчання η задається вручну людиною, що проводить навчання нейронної мережі. Загальна логіка оновлення значень ваг наступна: якщо вага збільшує помилку, то її значення необхідно зменшити і навпаки. Малі значення η призводять до повільного навчання, але надто великі можуть зробити навчання нестабільним. Типові значення η на практиці: 0,001, 0,0005, 0,0001.

5. ОСНОВНІ ПАРАМЕТРИ НАВЧАННЯ НЕЙРОННИХ МЕРЕЖ

Практична складова навчання нейронної мережі зводиться до налаштування низки важливих параметрів, основні з яких розглянуті нижче. І найбільш базовими із них є частково взаємопов'язані три параметри: епоха (epoch), батч (batch) та ітерація (iteration).

Епоха — один повний прохід нейронної мережі по всьому навчальному набору даних датасету. Наприклад, у випадку датасету, що складається із 10000 рядків, однією епохою буде вважатися аналіз моделлю всіх 10000 записів. Необхідність певної кількості епох пояснюється тим, що модель не може ефективно навчитися за одну епоху, що вимагає проведення повторних епох. Тому процес повторюється: $epoch_1 \rightarrow epoch_2 \rightarrow epoch_3 \rightarrow \ldots \rightarrow epoch_n$. В ідеальному випадку з кожною новою епохою помилка зменшується, ваги стають точнішими, а модель краще класифікує атаки.

Батч — розмір групи даних, яку нейронна мережа обробляє за один раз. Оскільки ціла епоха обробляється дуже повільно, її доцільно розділити на менші батчі. Таким чином, одна епоха складається із певної кількості рівних за розмірами батчів. Мала величина батчу призводить до стабільнішого навчання, але сам процес при цьому повільніший. І навпаки, при великому значенні батчу навчання відбувається швидше, але може бути менш стабільним. Типові значення батчу: 16, 32, 64, 128.

Важливо зауважити, що описане вище оновлення ваг відбувається після проходження кожного батчу, а один крок такого оновлення називається **ітерацією**.

Відповідно процес навчання має таку ієрархію: датасет → епоха → батч → запис.

Окрім того, при навчанні нейронної мережі необхідно вибрати алгоритм оптимізації та оцінювати метрики її ефективності.

Оптимізатор — алгоритм, що змінює ваги нейронної мережі для зменшення помилки. Базовим алгоритмом оптимізації, як вже було вказано вище, є градієнтний спуск.

Популярним оптимізатором є **Adam** (Adaptive Moment Estimation). Його перевагами є швидке навчання, стабільність та автоматичне налаштування швидкості навчання.

Двома важливими метриками ефективності роботи нейронної мережі є **точність (accuracy)** та **матриця сплутаних класів (confusion matrix)**.

Точність визначається наступною формулою відношенням правильних передбачень до загальної кількості передбачень. Наприклад, якщо із 100 записів в датасеті нейронна мережа зробила правильні передбачення для 87, то точність складатиме 87%.

Матриця сплутаних класів є показує, як мережа класифікує кожен клас і які класи «плутала» між собою. Приклад такої матриці наведено у таблиці 3.

	Передбачення A	**Передбачення B**	**Передбачення C**
Результат A	50	2	3
Результат B	4	45	1
Результат C	0	3	48

Таблиця 3. Приклад матриці сплутаних класів

6. ВИСНОВКИ

1. Сучасні мережі генерують мільйони подій щодня, що робить ручний аналіз або використання лише сигнатурних методів (пошук за шаблоном) неефективними проти нових і швидких атак.
2. Машинне навчання — це галузь штучного інтелекту, яка дозволяє системам автоматично знаходити закономірності в даних.
3. Нейронна мережа — це математична модель, що складається з шарів штучних нейронів та зв’язків між ними. Головним елементом навчання є налаштування ваг цих зв'язків.
4. Структура нейронної мережі:
   - вхідний шар — приймає ознаки (без обробки);
   - приховані шари — проводять основні обчислення через лінійну комбінацію (зважену суму) та активаційну функцію для виявлення складних закономірностей;
   - вихідний шар — формує вектор ймовірностей приналежності події мережевої події до певного класу.
5. Процес навчання включає пряме поширення сигналу, обчислення помилки та зворотне поширення помилки для оновлення ваг за допомогою градієнтного спуску.
6. Параметри навчання нейронної мережі регулюється кількістю епох (проходів по датасету) та розміром батчів (груп даних). Ефективність оцінюється за точністю та матрицею сплутаних класів.

7. ПИТАННЯ ДЛЯ САМОКОНТРОЛЮ

1. Чому традиційний сигнатурний підхід стає менш ефективним у сучасних умовах захисту мереж?
2. Сформулюйте визначення машинного навчання як галузі штучного інтелекту.
3. Поясніть різницю між керованим та некерованим навчанням?
4. У чому полягає перевага систем виявлення вторгнень (IDS) на основі аномалій порівняно з системами на основі сигнатур?
5. Поясніть поняття «перенавчання». Чому воно є критичною проблемою для моделі?
6. Для чого датасет розділяють на тренувальну та тестову частини? Опишіть роль кожної.
7. З яких трьох основних елементів складається штучна нейронна мережа?
8. Що таке «вага» зв'язку і як вона впливає на результат роботи нейрона?
9. Опишіть три типи шарів класичної нейронної мережі та їхні функції.
10. Як визначається кількість нейронів на вхідному та вихідному шарах для конкретної задачі?
11. Для чого при обчисленні зваженої суми використовується параметр зміщення?
12. Що таке активаційна функція і які основні завдання вона виконує в мережі?
13. Чому для багатокласової класифікації на вихідному шарі часто використовується саме активаційна функція Softmax?
14. Опишіть етапи алгоритму навчання нейронної мережі, починаючи від ініціалізації ваг.
15. Як працює метод зворотного поширення помилки?
16. Що таке «градієнтний спуск» і як він допомагає мінімізувати помилку моделі?
17. Як швидкість навчання впливає на стабільність тренування моделі?
18. Поясніть ієрархію: Датасет → Епоха → Батч → Ітерація.
19. Як розраховується метрика точності нейронної мережі?
20. Яку інформацію надає «матриця сплутаних класів» розробнику системи захисту?